#중급반 4주차

분리 집합, 최소 스패닝 트리

<Disjoint Set, Minimum Spanning Tree>

2p /<그래프/트리 용어>
6p /<분리 집합>
23p/<최소 스패닝 트리>
35p/<연습 문제>

#그래프/트리 용어

Definition

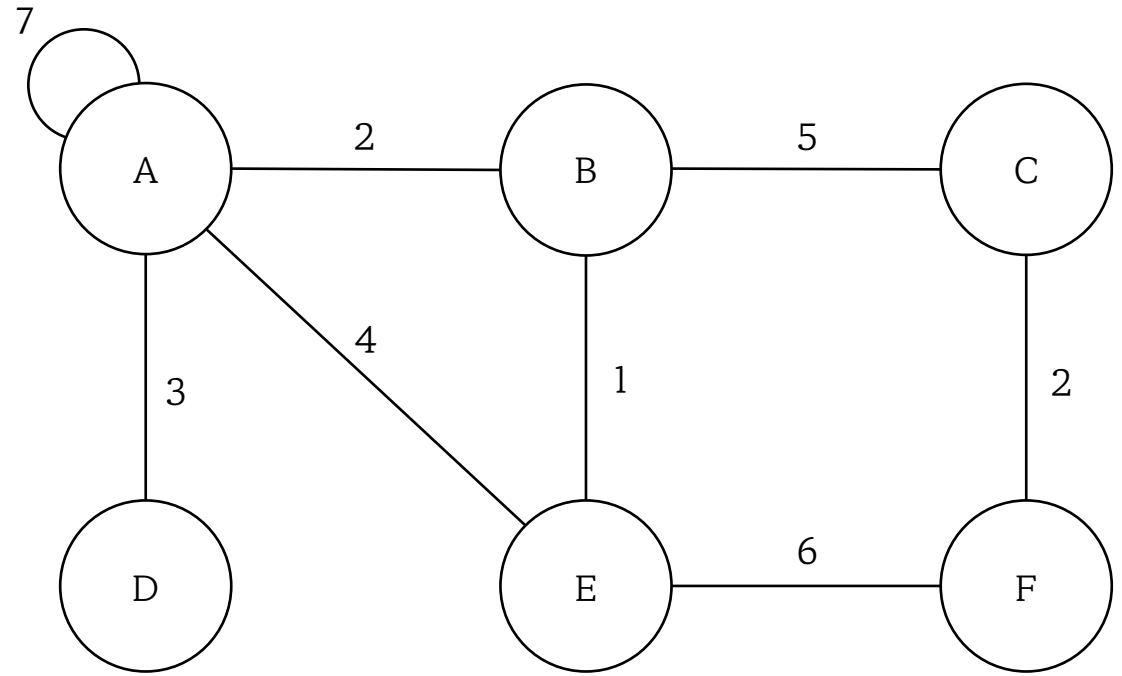
정점(노드) : 그래프를 이루는 객체(꼭짓점)
vertices(nodes)

간선 : 정점을 연결하는 선(변)
edges

루프 : 시작과 끝이 같은 간선
loop

가중치 : 그래프의 간선에 붙는 수치
weight

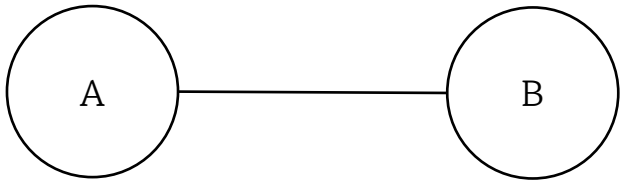
그래프 : 정점과 간선으로 이루어진 구조
graphs



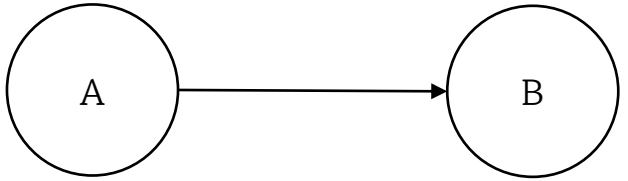
#그래프/트리 용어

Definition

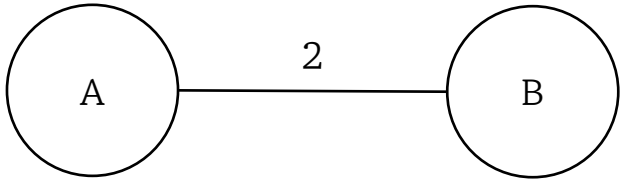
양방향 그래프 : 간선의 방향이 없는 그래프
undirected graph



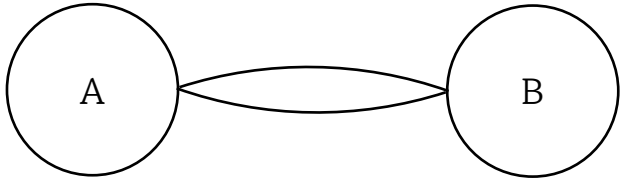
단방향 그래프 : 간선의 방향이 있는 그래프
directed graph(digraph)



가중치 그래프 : 간선에 가중치가 붙은 그래프
weighted graph



다중 그래프 : 두 정점을 잇는 간선이 여러 개일 수 있는 그래프
multigraph



#그래프/트리 용어

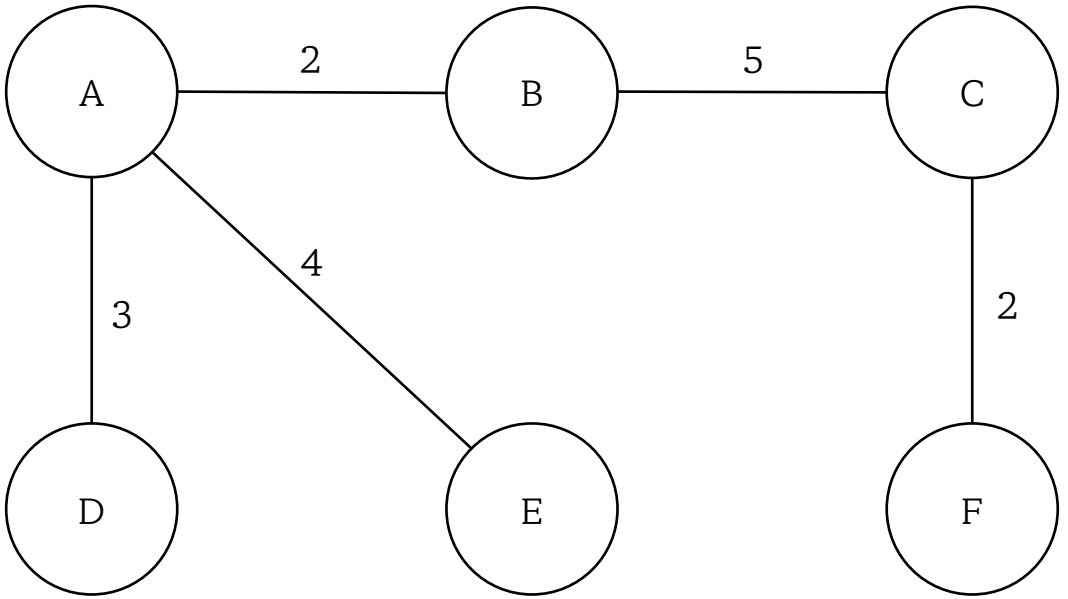
Definition

사이클 : 중복된 간선 없이 같은 정점으로 돌아오는 경로

cycle

트리 : 사이클이 없는 그래프

tree



#그래프/트리 용어

Definition

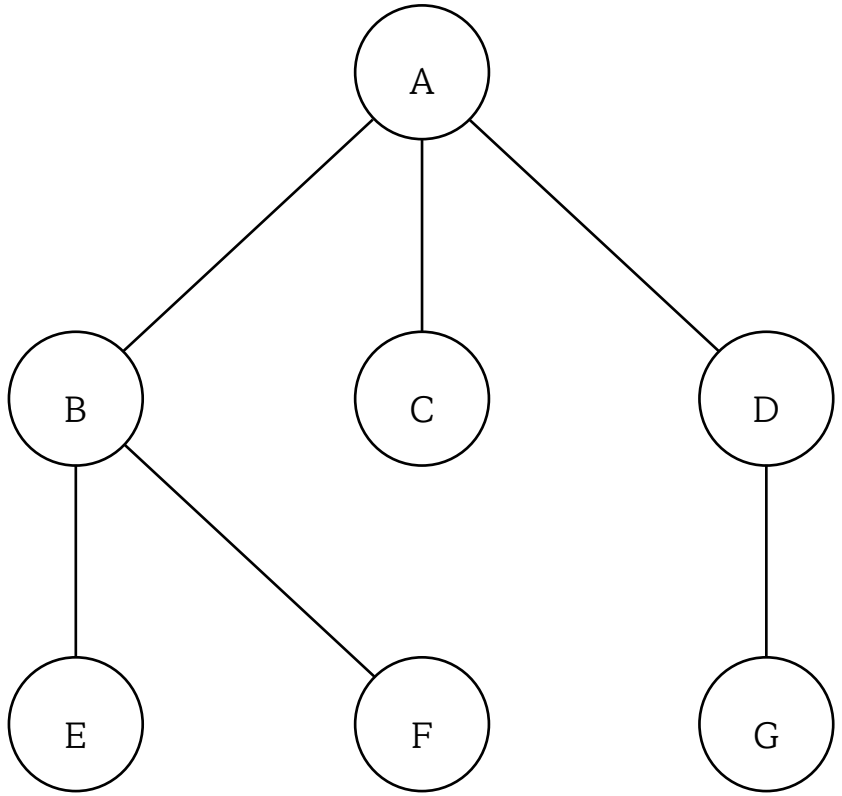
부모/자식 노드 : 어떤 노드의 상위/하위 노드
parent/child

루트 노드 : 부모 노드가 없는 최상위 노드
root node

단말 노드 : 자식 노드가 없는 최하위 노드
leaf node

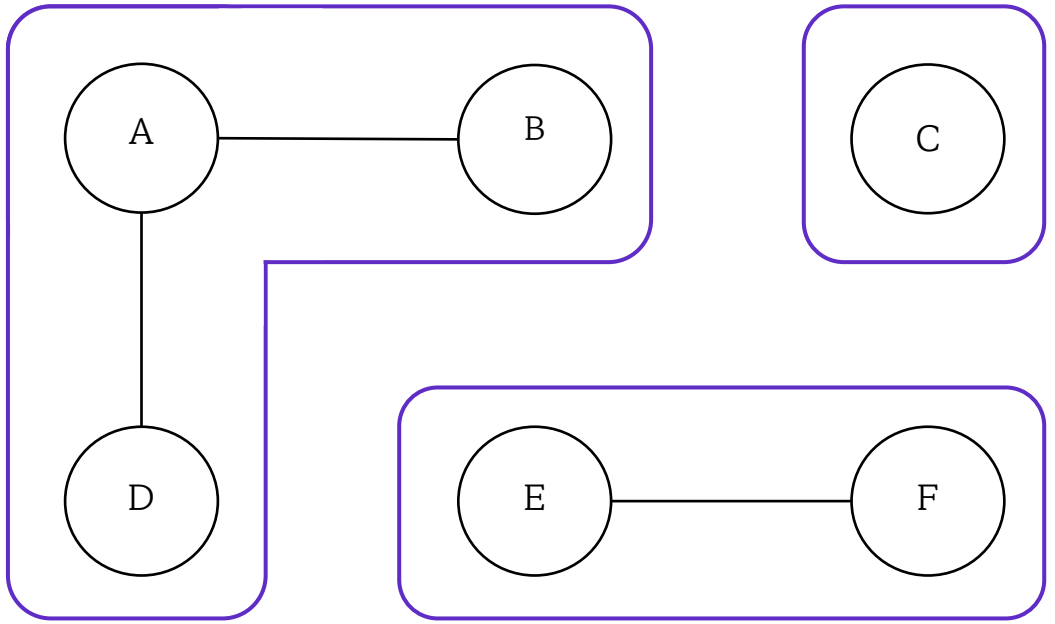
내부 노드 : 루트 노드와 단말 노드 사이의 노드
internal node

형제 : 같은 부모를 가지는 노드
siblings



#분리 집합
Disjoint Set

분리 집합 : 모든 원소를 겹치는 부분 없이 분리한 집합
disjoint Set

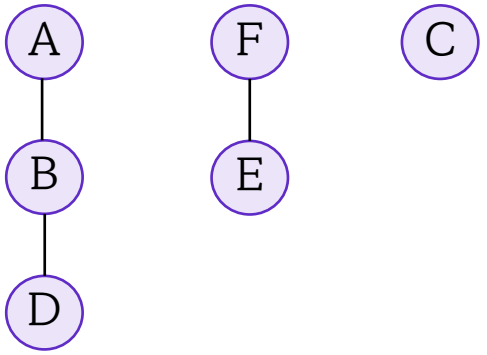


각 집합을 트리 형태로 저장하게 됩니다.

분리 집합을 구현하기 위해서 다음 두 함수가 필요합니다.

1. Find : 루트 노드 찾기

2. Union : 두 집합 합치기



#분리 집합
Disjoint Set

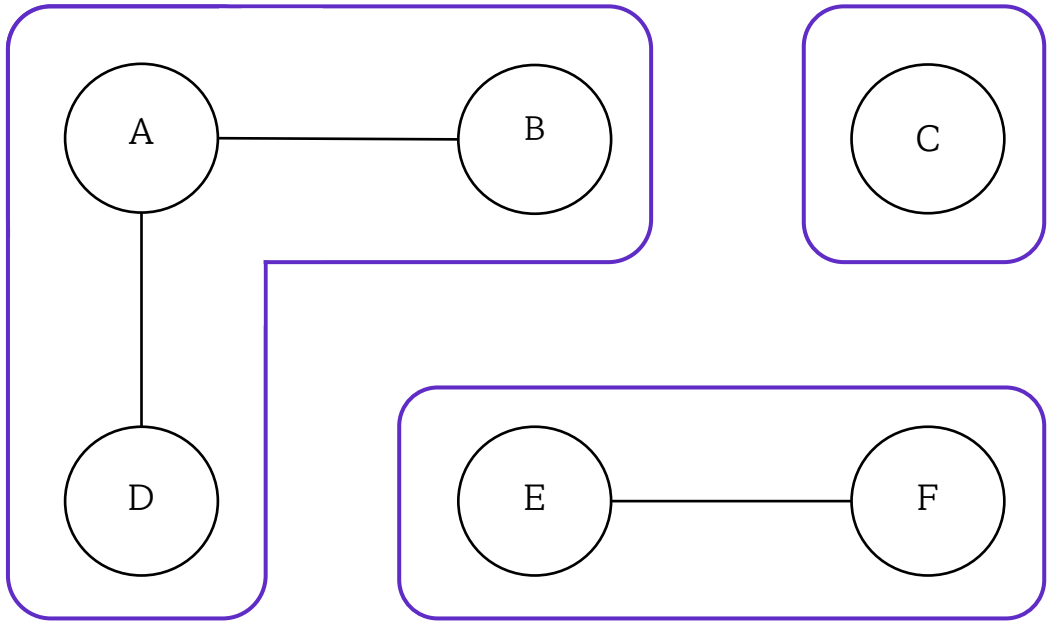
Find : 어떤 노드의 루트 노드 찾기
Finding root node

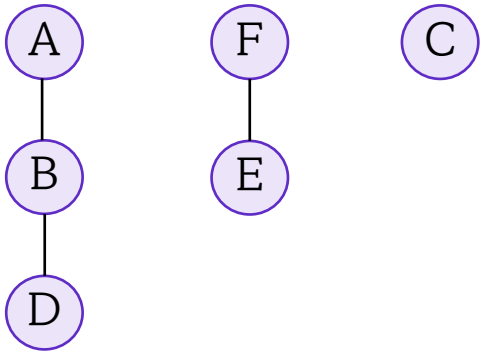
우선 트리를 저장하기 위해서 배열을 하나 선언합니다.

배열 'parent'에 각 노드의 부모 노드를 저장합니다.

만약 부모가 없으면 자기자신을 저장합니다.

처음에는 모든 노드가 자기 자신을 저장하고 있습니다.





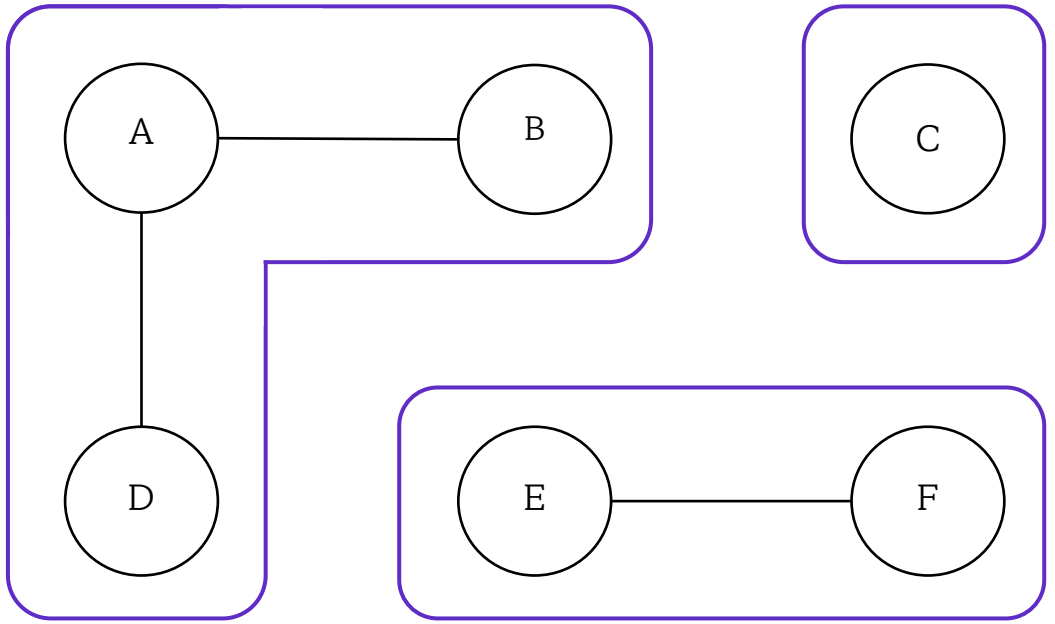
index	value
A	A
B	A
C	C
D	B
E	F
F	F

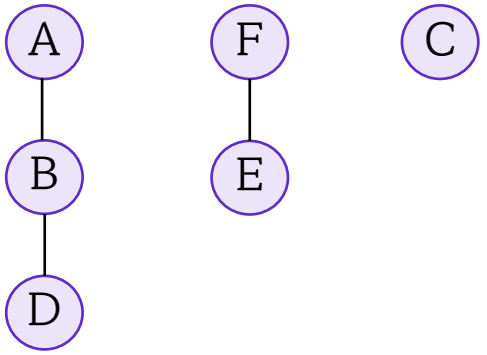
#분리 집합
Disjoint Set

Find : 어떤 노드의 루트 노드 찾기
Finding root node

루트를 찾는 것은 재귀 함수를 사용하면 됩니다.

1. index==value : 현재 index가 루트이므로 반환
2. index!=value: value(index의 부모)에 대하여 Find를 다시 적용





index	value
A	A
B	A
C	C
D	B
E	F
F	F

#분리 집합
Disjoint Set

Find : 어떤 노드의 루트 노드 찾기
Finding root node

Parent배열의 초기화와 Find의 구현은 다음과 같습니다.

init : parent를 초기화 하는 함수

Find : 루트 노드를 찾는 함수

```
int N;
vector<int> parent;

void init() {
    parent.resize(N + 1);
    for(int i=0; i <= N; i++) parent[i] = i; // 자기 자신으로 초기화
}

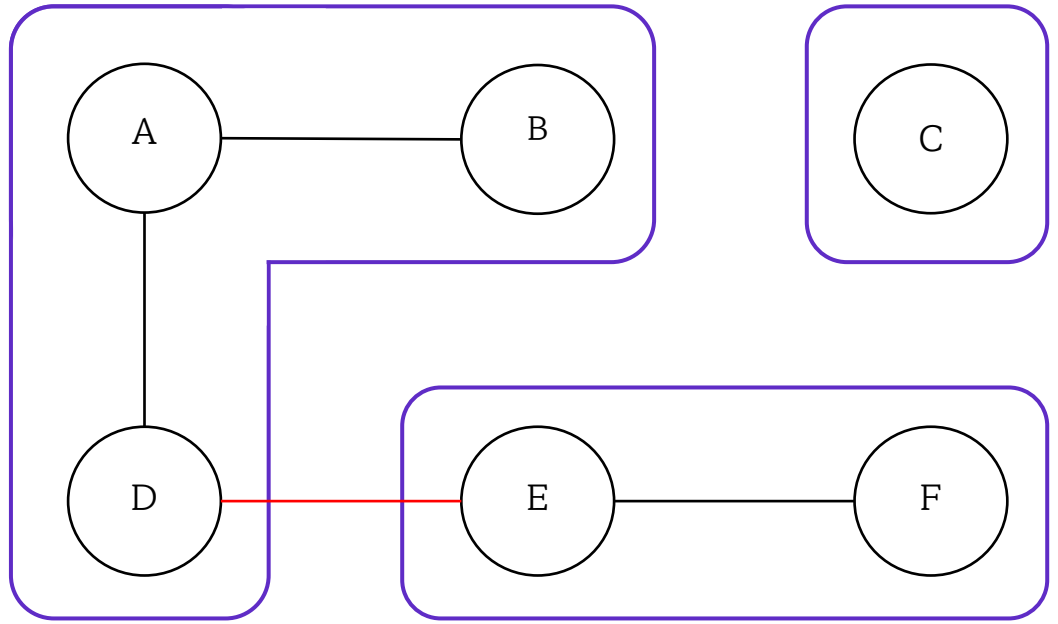
int Find(int x) {
    if (parent[x] == x) return x;
    return Find(parent[x]);
}
```

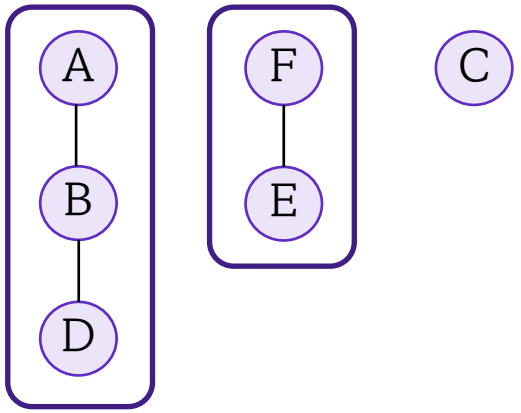
#분리 집합
Disjoint Set

Union : 다른 분리 집합을 합치기
Merging two disjoint set

노드 D, E를 연결하는 간선이 추가되었다고 가정합시다.

그러면 저희는 A,B,D 와 E,F 두 집합을 하나로 합쳐야 합니다.

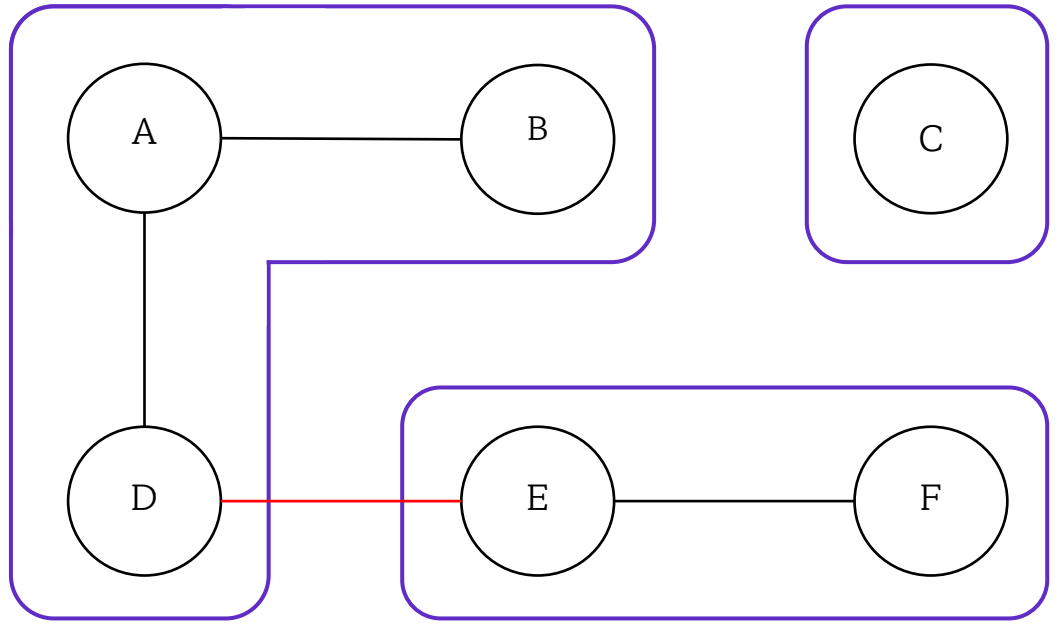




index	value
A	A
B	A
C	C
D	B
E	F
F	F

#분리 집합
Disjoint Set

Union : 다른 분리 집합을 합치기
Merging two disjoint set

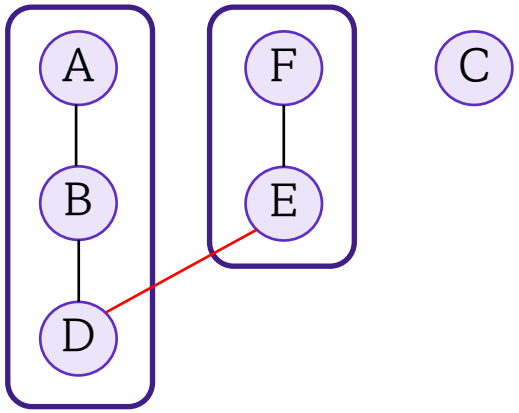


만약 parent[D]=E로 수정하게 되면 기존에 저장되어

B-D를 연결하는 간선의 정보가 소실됩니다.

마찬가지로 parent[E]=D로 해도 E-F의 정보 손실이 발생

하게 됩니다.



index	value
A	A
B	A
C	C
D	B
E	F
F	F

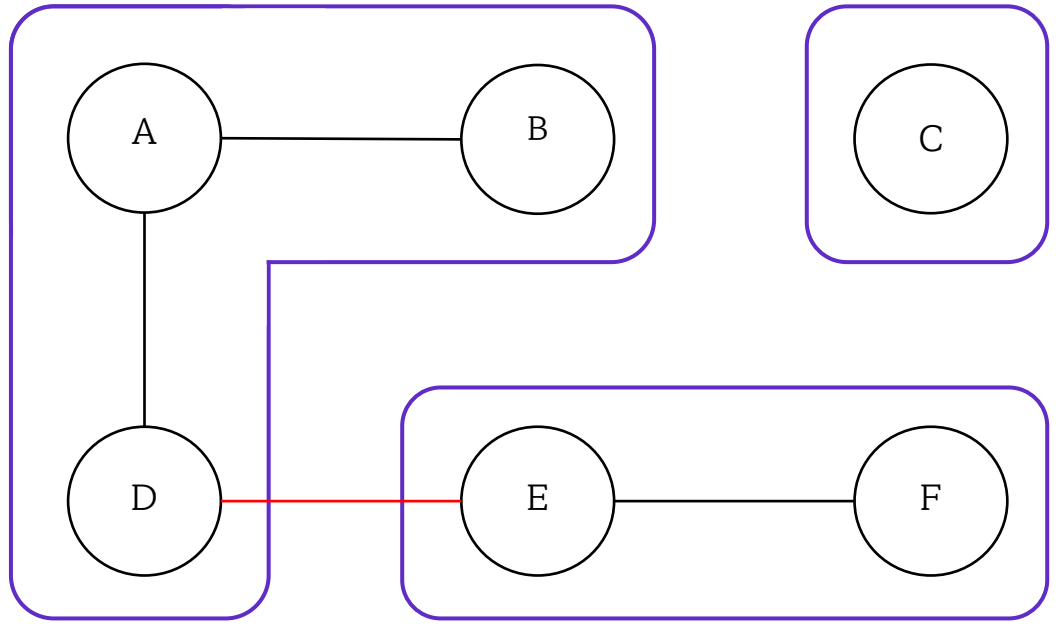
#분리 집합
Disjoint Set

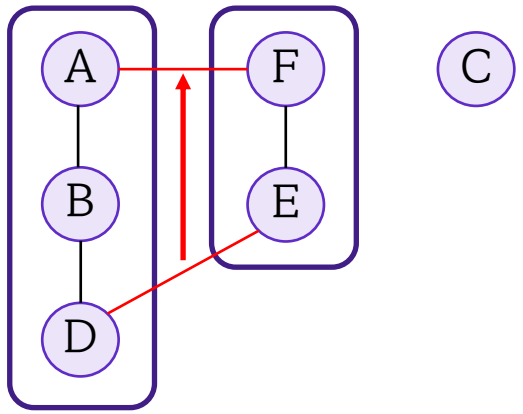
Union : 다른 분리 집합을 합치기
Merging two disjoint set

합치게 되었을 때 루트 노드는 유일하게 존재하게 됩니다.

합치기 이전의 상태에서는 각각의 트리에서 루트 노드가 하나씩 존재하여 총 두 개가 있습니다.

그렇기에 루트 노드 중 하나를 사용하여 두 집합의 연결성을 나타내는 정보를 저장할 수 있습니다.

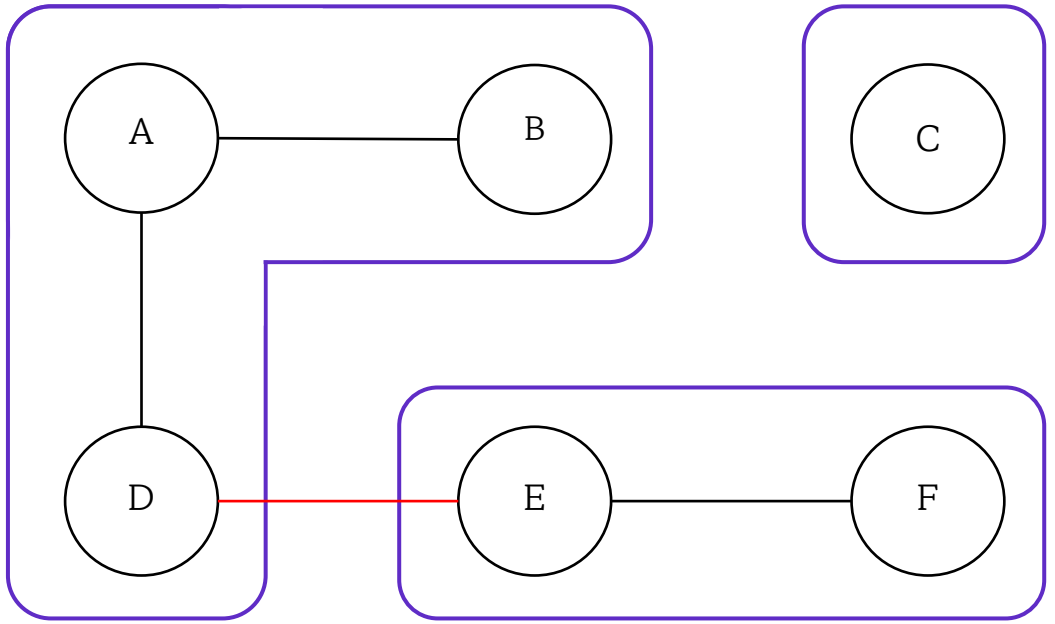




index	value
A	A
B	A
C	C
D	B
E	F
F	F

#분리 집합
Disjoint Set

Union : 다른 분리 집합을 합치기
Merging two disjoint set



이 과정을 위해서

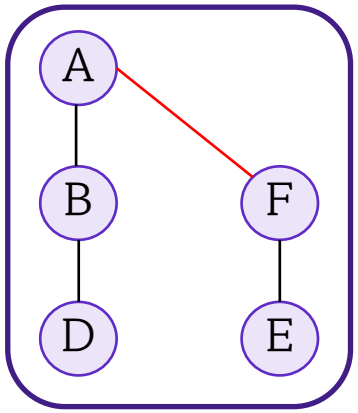
1. 각 트리의 루트 노드를 찾아야 합니다.

   이는 Find를 이용할 수 있습니다.

2. 정보 저장

   옆의 예시 같은 경우에는 parent[F]=A를 사용했지만

   parent[A]=F를 사용하는 것도 가능합니다.



C

index	value
A	A
B	A
C	C
D	B
E	F
F	A

#분리 집합
Disjoint Set

Union : 다른 분리 집합을 합치기
Merging two disjoint set

Union함수의 기본 구현은 다음과 같습니다.

노드x가 포함된 분리 집합과 , 노드 y가 포함된 분리집합을 하나로 합치는 함수입니다.

노드x, 노드y가 다른 분리집합에 있는 경우에만 합치게 됩니다.

```
void Union(int x, int y){
    x = Find(x);
    y = Find(y);
    if(x != y)
        parent[x] = y;
}
```

#분리 집합

Disjoint Set

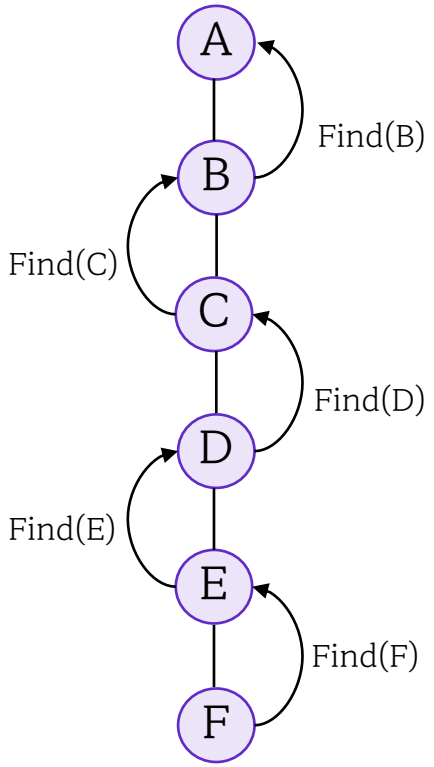
최적화 : 더 효율적으로 처리하기

Optimization

다음과 같이 선형적으로 저장된 트리를 봅시다.

Find(F)를 하게 되면 Find함수는 총 5번 호출됩니다.

이처럼 최악의 경우 (그래프 깊이)-1 번의 Find함수가 호출됩니다.



index	value
A	A
B	A
C	B
D	C
E	D
F	E

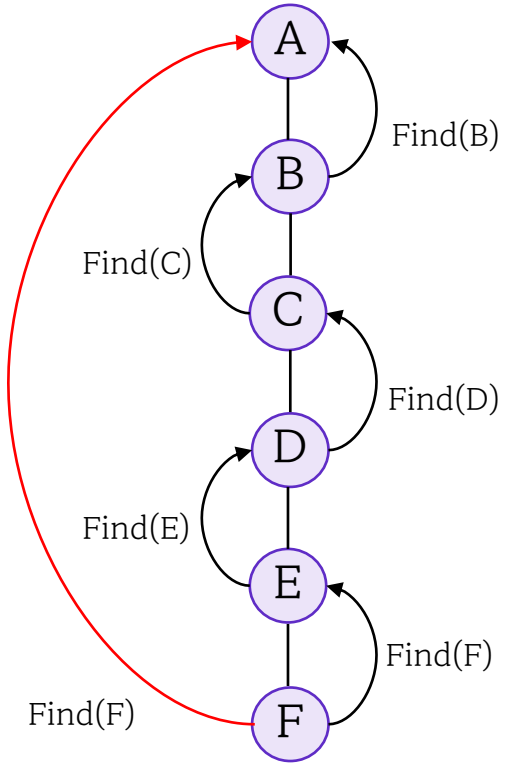
#분리 집합

Disjoint Set

최적화 : 더 효율적으로 처리하기

Optimization

Find(F)를 처음 시행할 때 parent[F]=A(루트 노드)로 업데이트하게 되면 이후 Find(F)를 하면 한번의 함수 호출만으로 루트를 찾을 수 있게 됩니다.



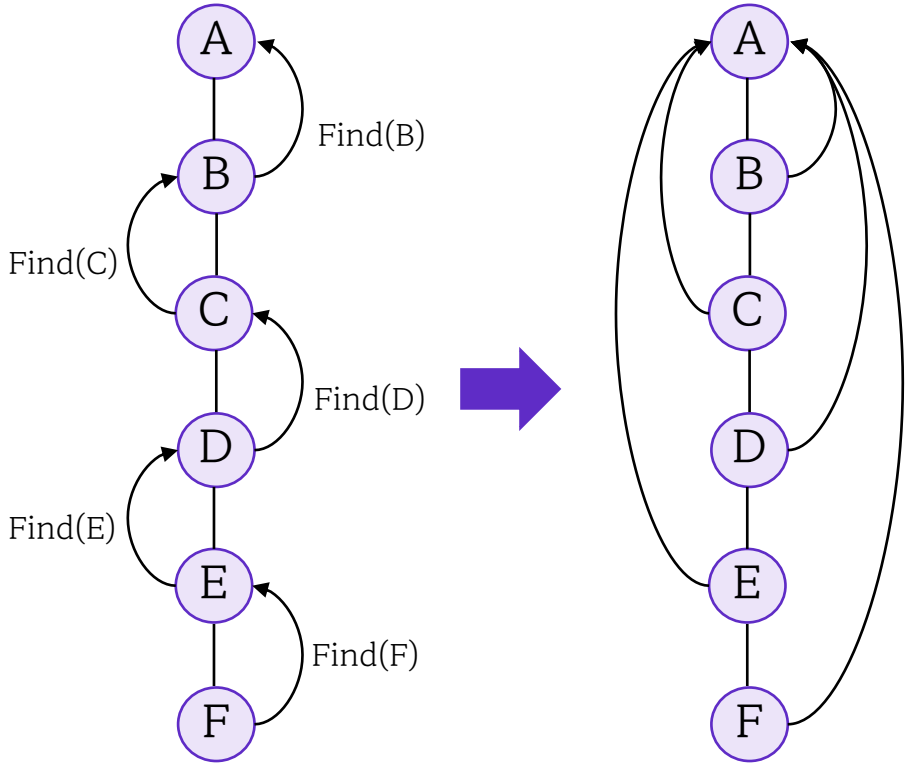
index	value
A	A
B	A
C	B
D	C
E	D
F	A

#분리 집합
Disjoint Set

최적화 : 더 효율적으로 처리하기
Optimization

또한, Find를 하며 올라왔던 parent들의 루트 노드도

동일하기에 모두 업데이트를 해줄 수 있습니다.

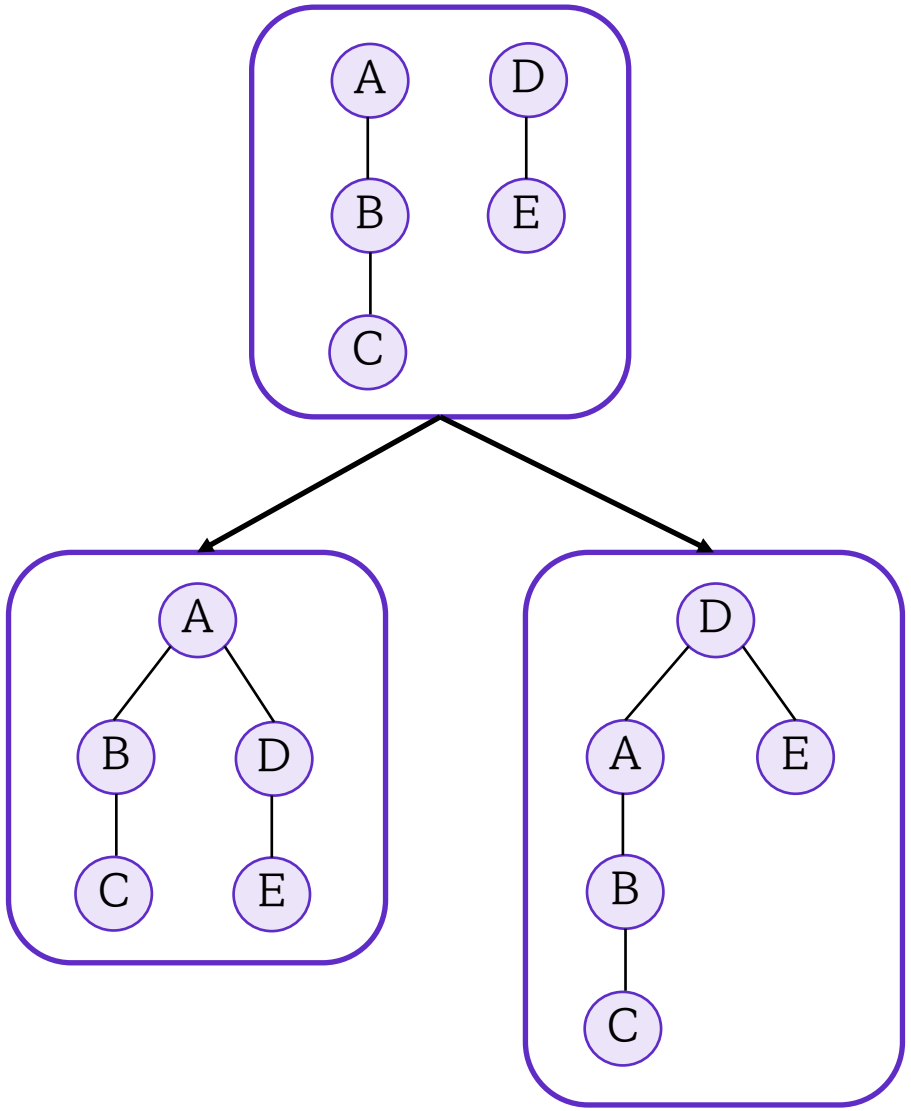


#분리 집합
Disjoint Set

최적화 : 더 효율적으로 처리하기
Optimization

다음의 두 집합을 하나로 합칠 때 두 가지 방법이 있습니다.

이중 무엇이 더 효율적일까요?

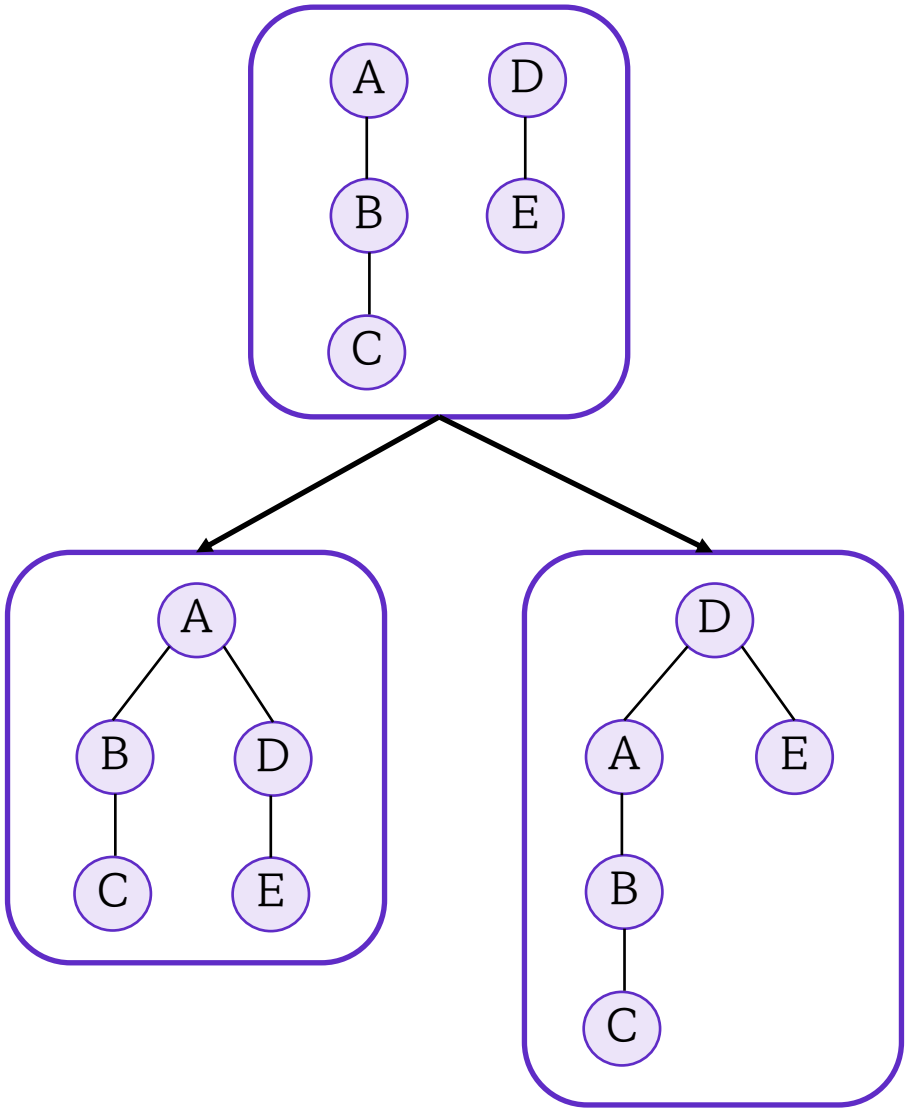


#분리 집합
Disjoint Set

최적화 : 더 효율적으로 처리하기
Optimization

왼쪽의 방법이 더 효율적입니다. 이는 최악의 경우 호출하는 Find함수의 횟수가 더 적기 때문입니다.

이처럼 depth가 더 큰 집합의 루트 노드를 전체 루트 노드로 사용하는 것이 더 효율적입니다.



#분리 집합
Disjoint Set

최적화 : 더 효율적으로 처리하기
Optimization

두 가지 최적화를 모두 반영한 코드입니다.

시간 복잡도는 아래와 같습니다.

최적화 없는 코드 : $O(N)$

둘 중 하나만 적용: $O(logN)$

모두 적용: $O(c)$(상수)

```
int N;
vector<int> parent, depth;

void init() {
    parent.resize(N + 1);
    depth.resize(N + 1);
    for(int i=0; i <= N; i++) parent[i] = i;
}

int Find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = Find(parent[x]); // 경로 압축
}

bool Union(int x, int y) {
    x = Find(x); y = Find(y);
    if(x == y) return false;

    // 트리 depth의 "상한"이 작은 트리를 큰 트리의 루트 아래에 붙임
    if(depth[x] < depth[y]) swap(x, y);
    parent[y] = x;

    // depth가 같을 경우에는, Union 연산 이후 트리 depth의 상한이 1 늘어남
    if(depth[x] == depth[y]) depth[x]++;

return true;
}
```

#분리 집합

Disjoint Set

종교 (JUNGOL #1863)

<문제 설명>

- 학교에 n명의 학생이 있다.
- 같은 종교를 믿는 두 사람의 정보가 m개 주어진다.
- 학생들이 믿고 있는 종교가 총 몇 가지인가?

<제약 조건>

- $1 \leq n \leq 50,000$
- $1 \leq m \leq 100,000$

#분리 집합
Disjoint Set

종교 (JUNGOL #1863)

<문제 해설>

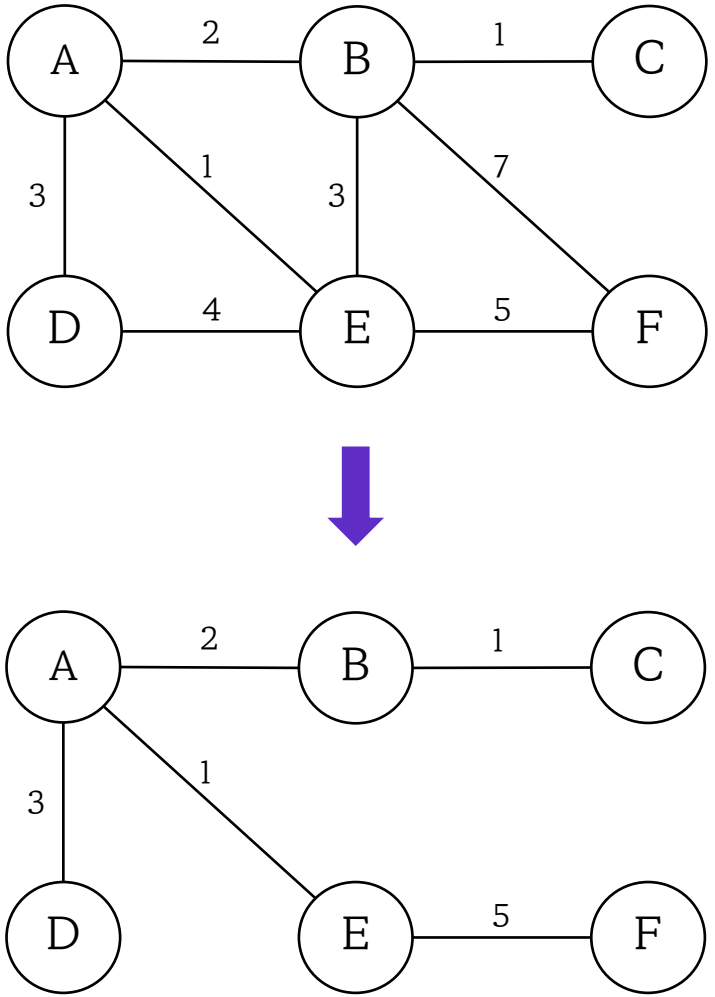
학생의 종교가 모두 다르다고 하면, 이때의 답은 N입니다.

쌍의 정보가 주어질 때마다 Union(x, y)를 실행하고, 합쳐질 때마다 답의 크기를 1씩 줄여나가면 됩니다.

#최소 스패닝 트리
Minimum spanning tree

스패닝 트리: 그래프의 모든 노드를 사이클 없이 포함하는 부분 그래프
spanning tree

최소 스패닝 트리(MST): 최소 비용의 스패닝 트리
minimum spanning tree



#최소 스패닝 트리

Minimum spanning tree

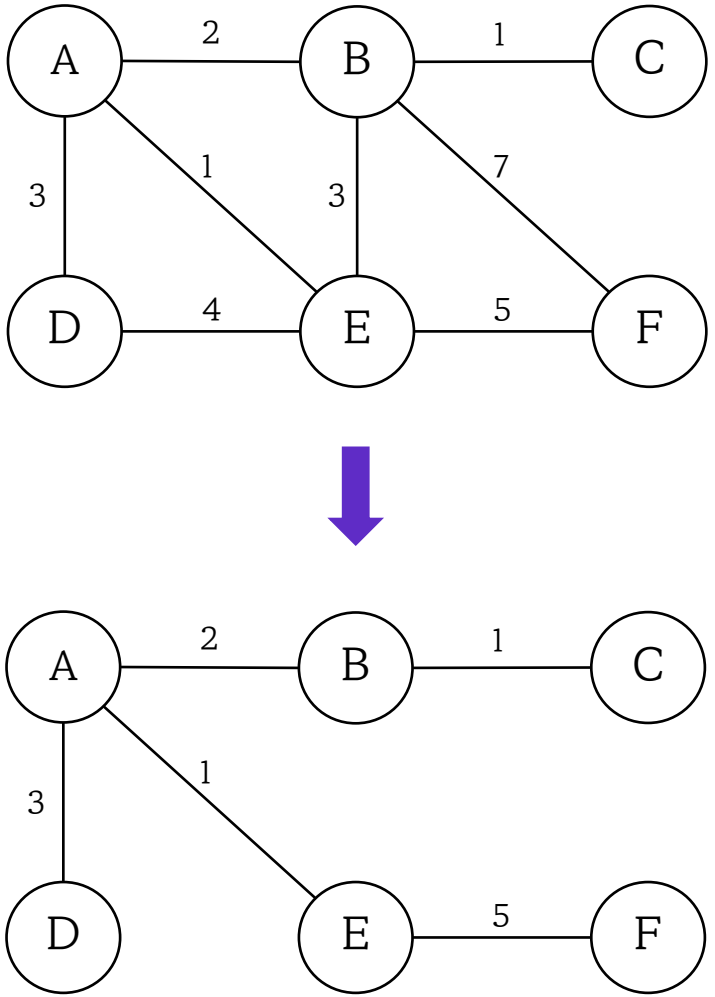
최소 스패닝 트리(MST): 최소 비용의 스패닝 트리

minimum spanning tree

MST를 찾는 방법은 두 가지 방법이 있습니다.

1. 크루스칼 알고리즘
2. 프림 알고리즘

두 방법 모두 공통적으로 가중치가 작은 간선을 우선적으로 사용하는 그리디 알고리즘을 사용합니다.



#최소 스패닝 트리

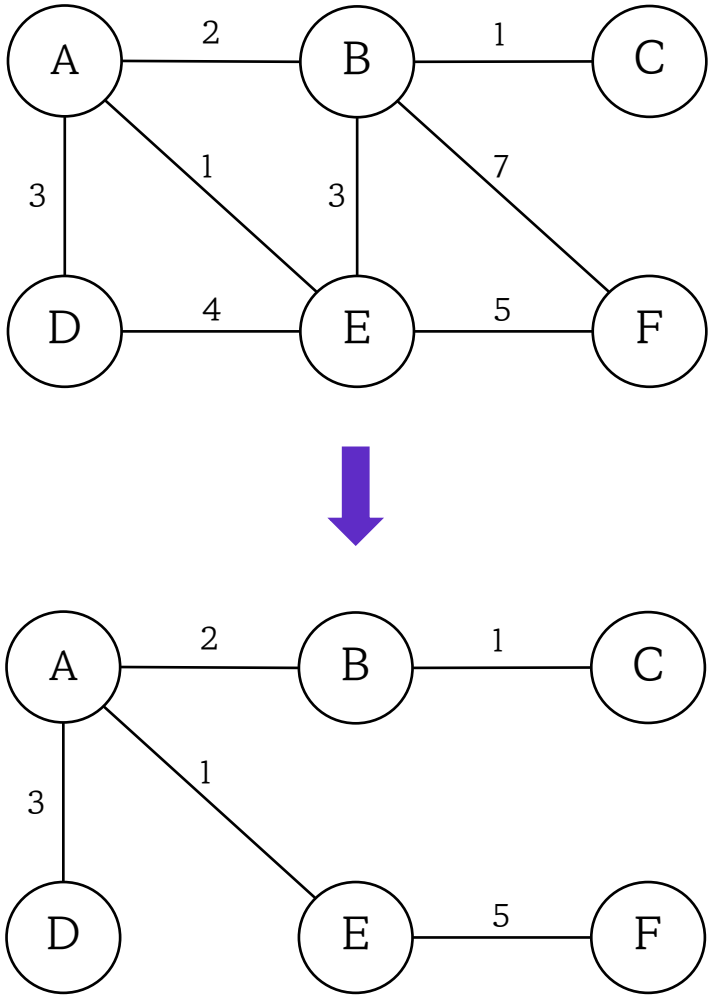
Minimum spanning tree

크루스칼 알고리즘

Kruskal's algorithm

노드만 있고 간선은 없는 시작상태에서 아래의
3단계의 과정으로 MST를 찾습니다.

1. 남은 간선 중 가장 가중치가 작은 간선 선택
2. 간선을 추가할 때 사이클이 발생하지 않으면 MST에 추가
3. 모든 간선에 대하여 1, 2반복



#최소 스패닝 트리

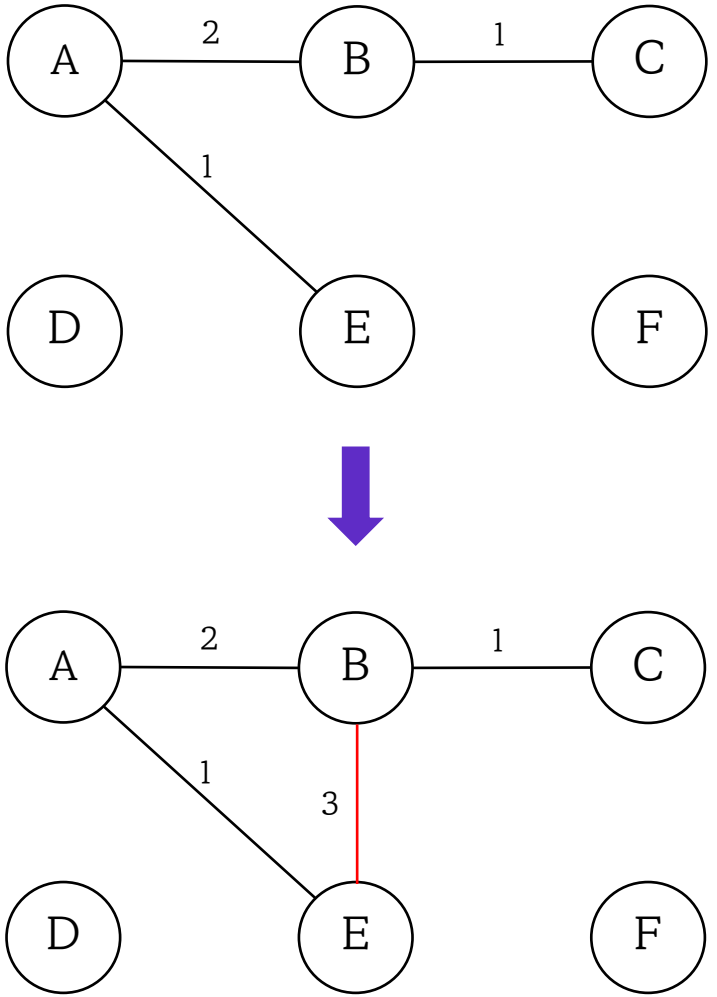
Minimum spanning tree

크루스칼 알고리즘

Kruskal's algorithm

단계2에서 사이클이 발생하게 되면 간선을 추가하지 않는 이유는 다음과 같습니다.

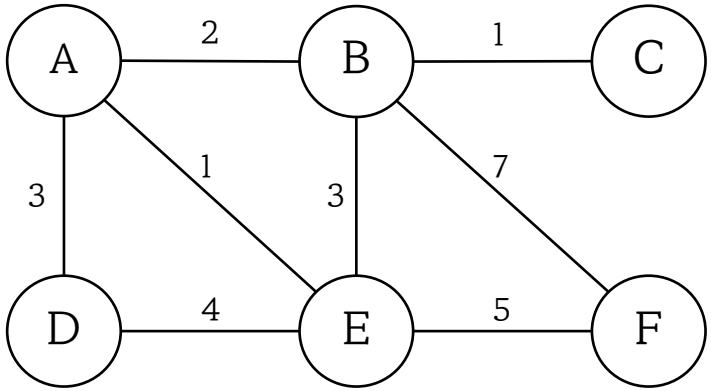
1. 트리가 아니게 됩니다. ->스패닝 트리의 조건을 만족하지 못함
2. 사이클내에서 이미 작거나 같은 가중치의 간선들로 서로 연결이 되어 있음

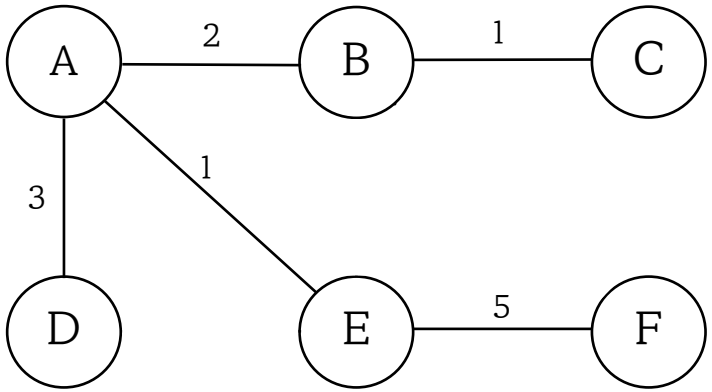


#최소 스패닝 트리
Minimum spanning tree

크루스칼 알고리즘
Kruskal's algorithm

크루스칼 알고리즘을 사용하기 위해서 우선 간선을 가중치 기준으로 정렬합니다. 이후 앞서 설명한 3단계를 거쳐 MST를 찾게 됩니다.





간선(가중치)
A-E(1)
B-C(1)
A-B(2)
B-E(3)
A-D(3)
D-E(4)
E-F(5)
B-F(7)

#최소 스패닝 트리

Minimum spanning tree

크루스칼 알고리즘을 사용하는 과정에서
분리 집합을 사용하게 됩니다.

정점 x와 정점 y가 같은 분리 집합에 있으면
x, y를 잇는 간선은 사이클을 이루게 됩니다.

기존의 Union 함수에서는,
사이클이 발생하게 되면 false를
사이클이 발생하지 않으면 true를
반환합니다.

```
bool Union(int x, int y) {
    x = Find(x); y = Find(y);
    if(x == y) return false;

    // 트리 depth의 "상한"이 작은 트리를 큰 트리의 루트 아래에 붙임
    if(depth[x] < depth[y]) swap(x, y);
    parent[y] = x;

    // depth가 같을 경우에는, Union 연산 이후 트리 depth의 상한이 1 늘어남
    if(depth[x] == depth[y]) depth[x]++;

return true;
}
```

#최소 스패닝 트리

Minimum spanning tree

분리 집합 함수를 활용하여

크루스칼 알고리즘을 구현하면 다음과 같습니다.

시간복잡도 $O(E \log E)$를 가지게 됩니다.

```cpp
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    int M;
    cin >> N >> M;
    init();

    vector<tuple<int,int,int>> E; // {weight, from, to}
    for(int i=1,u,v,w; i <= M; i++){
        cin >> u >> v >> w;
        E.push_back(make_tuple(w, u, v));
    }
    sort(E.begin(), E.end()); // weight 기준 오름차순 정렬

    long long res = 0;
    for(int i=0; i < M; i++){
        int w = get<0>(E[i]), u = get<1>(E[i]), v = get<2>(E[i]);
        if(Union(u, v)) res += w;
    }
    cout << res;
}
```

#최소 스패닝 트리

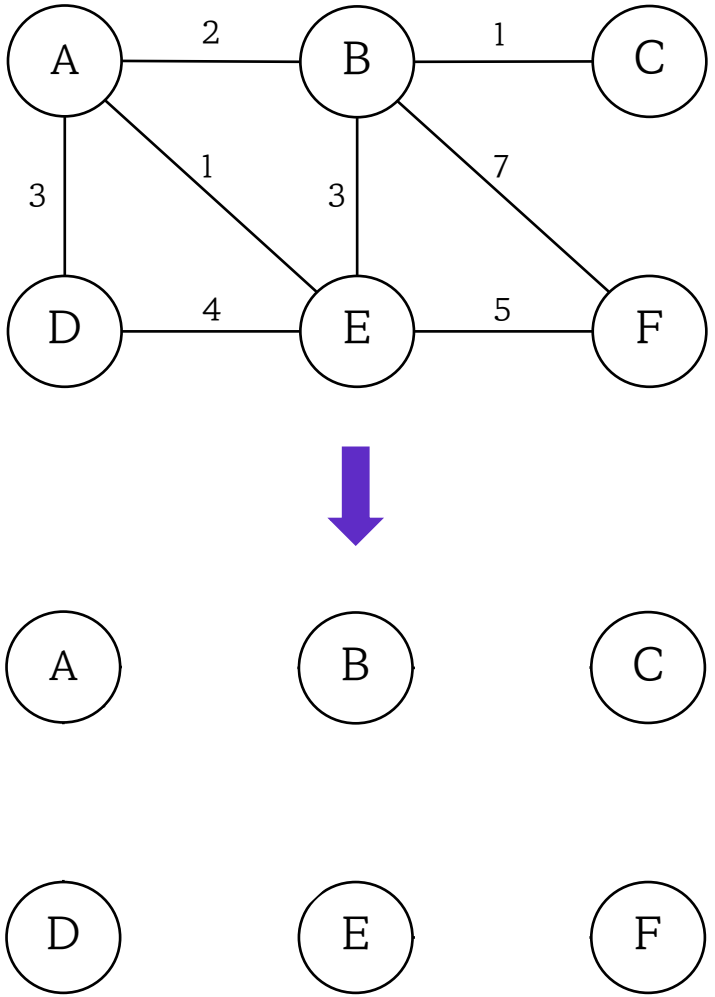
Minimum spanning tree

프림 알고리즘

Prim's algorithm

마찬가지로 3단계의 과정으로 MST를 찾습니다.

1. 임의의 정점 하나를 추가.
2. 이미 연결된 정점에서 도달할 수 있는 나머지 정점 중
   가중치가 가장 작은 간선으로 연결되는 정점을 MST에 추가한다.
3. 모든 정점이 트리에 있을 때까지 2반복



#최소 스패닝 트리

Minimum spanning tree

프림 알고리즘을 구현할 때 priority_queue를 사용합니다.

(가중치, 도달 가능 정점)의 형식으로 넣은 후 ,
가중치가 가장 작은 데이터를 우선적으로 빼내면서 정점을 방문하지 않았으면
MST에 계속 추가하는 과정을 반복합니다.

시간복잡도 $O(E \log V)$를 가지게 됩니다.

#최소 스패닝 트리

Minimum spanning tree

```cpp
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    int M;
    cin >> N >> M;

    // adj[u] = {weight, to}
    vector<vector<pair<int,int>>> adj(N + 1);
    vector<bool> visited(N + 1, false);

    for(int i=1,u,v,w; i <= M; i++){
        cin >> u >> v >> w;
        adj[u].push_back({w, v});
        adj[v].push_back({w, u}); // 양방향 그래프
    }

    // {weight, 정점} 을 weight 기준 오름차순으로 꺼내는 최소 힙
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, 1}); // 임의의 정점(1번)에서 시작

    long long res = 0;
    int cnt = 0; // MST에 포함된 정점 수
    while(!pq.empty() && cnt < N){
        int w = pq.top().first, u = pq.top().second;
        pq.pop();
        if(visited[u]) continue; // 이미 MST에 포함된 정점이면 무시

        visited[u] = true;
        res += w;
        cnt++;

        // 새로 추가된 정점에서 뻗어 나가는 간선을 후보로 추가
        for(int i=0; i < (int)adj[u].size(); i++){
            int nw = adj[u][i].first, v = adj[u][i].second;
            if(!visited[v]) pq.push({nw, v});
        }
    }
    cout << res;
}
```

#최소 스패닝 트리

Minimum spanning tree

최대신장트리 (JUNGOL #1350)

<문제 설명>

- 정점 V개와 간선 E개로 이루어진 그래프가 주어진다.
- 각 간선에는 가중치가 존재한다.
- 해당 그래프의 “최대” 신장 트리의 가중치 합을 구한다.

<제약 조건>

- $1 \leq V \leq 1{,}000$
- $1 \leq E \leq 20{,}000$

#예제

5 종교 (JUNGOL #1863)

분리집합 기본 문제

4 최대신장트리 (JUNGOL #1350)

MST 기본 문제

#연습 문제

5 종교 2 (JUNGOL #8406)

Union의 크기를 구해보자

4 최소비용신장트리 (JUNGOL #1060)

입력 형식이 바뀌어도 본질은 같아요

3 입소문의 힘 (JUNGOL #26276)

크루스칼 알고리즘의 원리는?

2 차량 모듈 제작 (JUNGOL #9752)

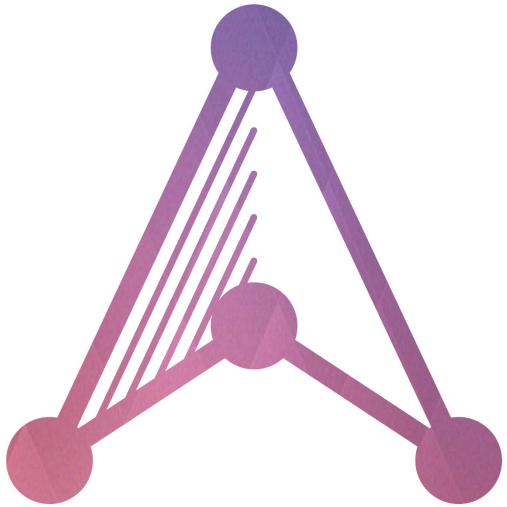
UCPC 2023 예선 C번

1 터널 (JUNGOL #2246)

이 문제는 조금 무식하게 풀어도 되지만…

5 행성 터널 (JUNGOL #6154)

MST를 이루는 터널 후보는 어떤 성질을 가지고 있을까요?



A L O H A

The algorithm club.